

PySpark Window Functions:

A Practical Guide for Data Engineers

Window Functions

Not all comparisons in analytics happen row-by-row. Sometimes, you must look at the bigger picture — and that's where Window Functions transform your dataset from plain tables into meaningful insights.

What are Window Functions?

Window functions in Spark let you perform calculations across groups of rows without collapsing your data like GROUP BY does.

They help answer questions like:

- What is the running total per customer?
- Who are the top 3 sellers per region?
- What is the difference from the previous transaction?
- What is the average sales in the last 7 days?

Define a window using:

PARTITION BY → How to group data
ORDER BY → Sorting order within each group
ROWS/RANGE → Extent of the sliding window (optional)

Quick setup:

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

1) row_number()

Purpose: Assign a unique sequence number per group.

Real-world example: Pick the latest record per customer from a transactions history.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w = Window.partitionBy("customer_id").orderBy(F.col("transaction_date").desc())

latest = (
    df
    .withColumn("rn", row_number().over(w))
    .filter("rn = 1")
    .drop("rn")
)
```

Note: Use a deterministic ORDER BY, often .desc() to get the latest.

2) rank()

Purpose: Rank items; ties get the same rank and ranks may skip numbers.

Example: Rank salespersons per region based on sales.

```
from pyspark.sql.functions import rank

w = Window.partitionBy("region").orderBy(F.col("sales").desc())
ranked = df.withColumn("rank", rank().over(w))
```

3) dense_rank()

Purpose: Like rank, but without gaps in the ranking numbers.

Example: Award top 3 performers in each team.

```
from pyspark.sql.functions import dense_rank

w = Window.partitionBy("team").orderBy(F.col("performance_score").desc())
df_top = df.withColumn("dense_rank", dense_rank().over(w)).filter("dense_rank <= 3")
```

4) lag()

Purpose: Access a previous row value.

Example: Difference in daily stock price compared to the previous day.

```
from pyspark.sql.functions import lag

w = Window.partitionBy("stock_id").orderBy("date")
with_prev = df.withColumn("prev_price", lag("price", 1).over(w))
with_diff = with_prev.withColumn("difference", F.col("price") - F.col("prev_price"))
```

Tip: Provide a default in lag(col, 1, default) to avoid nulls on the first row.

5) lead()

Purpose: Access a next row value.

Example: Compare today's order quantity with tomorrow's scheduled order.

```
from pyspark.sql.functions import lead

w = Window.partitionBy("item_id").orderBy("date")
next_q = df.withColumn("next_day_qty", lead("order_qty", 1).over(w))
```

6) sum(), avg(), etc. with a window

Purpose: Running or rolling totals.

Example: Cumulative sales per customer.

```
from pyspark.sql.functions import sum as sum_ # avoid shadowing built-in

w = (Window
      .partitionBy("customer_id")
      .orderBy("transaction_date")
      .rowsBetween(Window.unboundedPreceding, Window.currentRow))

with_running = df.withColumn("running_total", sum_("amount").over(w))
```

7) ntile(n)

Purpose: Divide each partition into n buckets of (approximately) equal size.

Example: Segment customers into 4 spending tiers.

```
from pyspark.sql.functions import ntile

w = Window.partitionBy("region").orderBy(F.col("total_spend").desc())
with_quartile = df.withColumn("spend_quartile", ntile(4).over(w))
```

Why Window Functions Matter for Data Engineers

- Used in slowly changing dimensions (SCD) handling
- Critical for audit and change tracking
- Common in financial, telecom, healthcare, and retail analytics
- Frequently asked about in Data Engineer interviews

Handy tips

- Always specify a clear ORDER BY for deterministic results
- For "so far up to this row," cap frames with `rowsBetween(unboundedPreceding, currentRow)`
- Repartition by partition keys before very heavy window operations